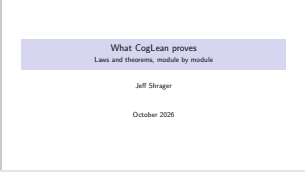
What CogLean proves

Laws and theorems, module by module

Jeff Shrager

October 2026

What CogLean proves



This is a reference deck: for each module of CogLean, which theory-neutral laws it has or lacks, and its headline theorems. It's drawn from a catalog generated from the Lean code itself, so it can't claim more than is proved. Two distinctions run through it: laws versus a module's own theorems, and general results versus single computed cases.

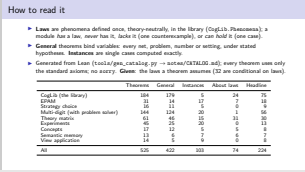
How to read it

- **Laws** are phenomena defined once, theory-neutrally, in the library (`CogLib.Phenomena`); a module *has* a law, *never* has it, *lacks* it (one counterexample), or *can hold* it (one case).
- **General** theorems bind variables: every net, problem, number or setting, under stated hypotheses. **Instances** are single cases computed exactly.
- Generated from Lean (`tools/gen_catalog.py` → `notes/CATALOG.md`); every theorem uses only the standard axioms; no `sorry`. **Given**: the laws a theorem assumes (32 are conditional on laws).

	Theorems	General	Instances	About laws	Headline
CogLib (the library)	184	179	5	24	75
EPAM	31	14	17	7	18
Strategy choice	16	11	5	0	9
Multi-digit (with problem solver)	144	124	20	1	56
Theory matrix	61	46	15	31	30
Experiments	45	25	20	0	13
Concepts	17	12	5	5	8
Semantic memory	13	6	7	6	7
View application	14	5	9	0	8
All	525	422	103	74	224

What CogLean proves

How to read it


How to read it

- **Laws** are phenomena defined once, theory-neutrally, in the library (CogLib.Phenomena): a module has a law, never has it, lacks it (one counterexample), or can hold it (one case).
- **General** theorems bind variables: every net, problem, number or setting, under stated hypotheses. **Instances** are single cases computed exactly.
- Generated from Lean (`tools/gen_catalog.py` → `notes/CATALOG.md`): every theorem uses only the standard axioms; no `sorry`. **Given**: the laws a theorem assumes (52 are conditional on laws).

	Theorems	General	Instances	About laws	Headline
CogLib (the library)	184	179	5	24	76
EPAM	31	14	17	7	18
Strategy choice	106	71	35	0	6
Multi-digit (with problem solver)	144	124	20	0	96
Chunk theory	40	36	4	31	7
Experiments	62	21	20	0	13
Concepts	17	12	5	5	8
Semantic memory	13	8	5	6	2
View application	14	7	7	1	2
All	525	432	103	74	224

How to read it. Laws are phenomena defined once in the library without reference to any theory, and a module has a law, never has it, lacks it by a single counterexample, or can hold it in a single case. General theorems hold for every net, problem, number or setting, under their stated hypotheses; instances are single cases computed exactly. The whole thing is generated from Lean, every theorem uses only the standard axioms, and there's no sorry anywhere. In all, five hundred twenty-five theorems, four hundred twenty-two of them general, and seventy-four about laws. The catalog also records which theorems assume a law among their premises, thirty-two of them, so a conditional result can't pass for a derived one.

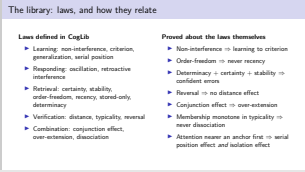
The library: laws, and how they relate

Laws defined in CogLib

- Learning: non-interference, criterion, generalization, serial position
- Responding: oscillation, retroactive interference
- Retrieval: certainty, stability, order-freedom, recency, stored-only, determinacy
- Verification: distance, typicality, reversal
- Combination: conjunction effect, over-extension, dissociation

Proved about the laws themselves

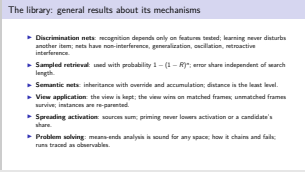
- Non-interference $\Rightarrow$ learning to criterion
- Order-freedom $\Rightarrow$ never recency
- Determinacy + certainty + stability $\Rightarrow$ confident errors
- Reversal $\Rightarrow$ no distance effect
- Conjunction effect $\Rightarrow$ over-extension
- Membership monotone in typicality $\Rightarrow$ never dissociation
- Attention nearer an anchor first $\Rightarrow$ serial position effect *and* isolation effect



The library defines the laws: of learning, of responding, of retrieval, of verification, and of conceptual combination. And some theorems are about the laws themselves, independent of any theory. Non-interference gives learning to criterion. Order-freedom rules out recency. Determinacy, certainty and stability together give confident errors. A reversal rules out the distance effect. The conjunction effect gives over-extension. And if membership is a nondecreasing function of typicality, the two can never dissociate. These are the reusable results: any theory that has the premises inherits the conclusion.

The library: general results about its mechanisms

- **Discrimination nets**: recognition depends only on features tested; learning never disturbs another item; nets have non-interference, generalization, oscillation, retroactive interference.
- **Sampled retrieval**: used with probability $1 - (1 - R)^n$; error share independent of search length.
- **Semantic nets**: inheritance with override and accumulation; distance is the least level.
- **View application**: the view is kept; the view wins on matched frames; unmatched frames survive; instances are re-parented.
- **Spreading activation**: sources sum; priming never lowers activation or a candidate's share.
- **Problem solving**: means-ends analysis is sound for any space; how it chains and fails; runs traced as observables.



The library also proves general results about each of its mechanisms. Discrimination nets: recognition depends only on the features tested, learning never disturbs another item, and nets have the learning and responding laws. Sampled retrieval: its closed forms. Semantic nets: inheritance, with override for single-valued properties and accumulation for many-valued ones. View application: the view is kept, the view wins on matched frames, everything else survives, and instances are re-parented. Spreading activation: activation from sources sums, and priming never lowers it. And problem solving: an impasse is exactly a violated precondition.

EPAM, and memory families against retrieval laws

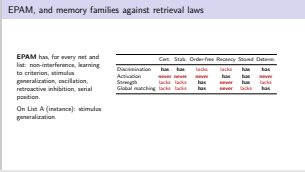
EPAM has, for every net and list: non-interference, learning to criterion, stimulus generalization, oscillation, retroactive inhibition, serial position.

On List A (instance): stimulus generalization.

	Cert.	Stab.	Order-free	Recency	Stored	Determ.
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	**never**	**never**	**never**	**has**	**has**	**never**
Strength	lacks	lacks	**has**	**never**	**has**	lacks
Global matching	lacks	lacks	**has**	**never**	lacks	**has**

What CogLean proves

EPAM, and memory families against retrieval laws



EPAM has all six of its classic phenomena for every net and list: non-interference, learning to criterion, stimulus generalization, oscillation, retroactive inhibition and the serial position effect, with stimulus generalization also shown on its List A. And the theory matrix: four memory families against six retrieval laws. Has and never are for every setting; lacks is a single counterexample, so those eight cells are the weakest results in the table.

Strategy choice (Siegler & Shrager 1984)

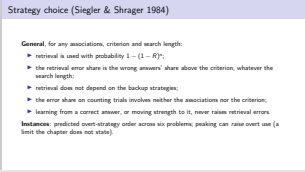
General, for any associations, criterion and search length:

- retrieval is used with probability $1 - (1 - R)^n$;
- the retrieval error share is the wrong answers' share above the criterion, whatever the search length;
- retrieval does not depend on the backup strategies;
- the error share on counting trials involves neither the associations nor the criterion;
- learning from a correct answer, or moving strength to it, never raises retrieval errors.

Instances: predicted overt-strategy order across six problems; peaking can *raise* overt use (a limit the chapter does not state).

What CogLean proves

Strategy choice (Siegler & Shrager 1984)



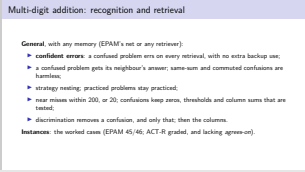
Strategy choice. In general, for any associations, criterion and search length: retrieval's probability of use, the retrieval error share, independence from the backup strategies, the counting error share, which is the chapter's nonintuitive prediction, and that strengthening the correct answer never raises errors. And two computed cases: the predicted order of overt strategy use across six problems, and a case where making a distribution more peaked raises overt strategy use, a condition the original chapter doesn't state.

Multi-digit addition: recognition and retrieval

General, with any memory (EPAM's net or any retriever):

- **confident errors**: a confused problem errs on every retrieval, with no extra backup use;
- a confused problem gets its neighbour's answer; same-sum and commuted confusions are harmless;
- strategy nesting; practiced problems stay practiced;
- near misses within 200, or 20; confusions keep zeros, thresholds and column sums that are tested;
- discrimination removes a confusion, and only that; then the columns.

Instances: the worked cases (EPAM 45/46; ACT-R graded, and lacking *agrees-on*).



Multi-digit addition, the recognition and retrieval side. In general, with any memory: confident errors, the neighbour's answer, harmless confusions when sums agree, strategy nesting, practiced problems staying practiced, near misses, and what confusions preserve when features are tested. And learning by discrimination removes exactly the confusion it targets. The worked cases are instances: forty-five in forty-six with EPAM, and graded confusion with ACT-R, which lacks the agrees-on property in that case.

Multi-digit addition: the column backup as problem solving

General, for every problem:

- **the bridge**: the problem solver answers exactly as the column backup, for every single-digit process;
- place value: carrying adds no error; a column error is a single-digit error moved to its place;
- without the carry rule: exact when no carry is needed; under strict critics, the no-carry bug;
- GPS works units first and answers exactly; without the carry operator it gives up on exactly the carry problems;
- **the same deficit, two regimes**: a confident bug under repair, no answer under GPS.

Instance: four repairs, four bugs on $15 + 17$.

Multi-digit addition: the column backup as problem solving

General, for every problem:

- **the bridge**: the problem solver answers exactly as the column backup, for every single-digit process;
- place value: carrying adds no error; a column error is a single-digit error moved to its place;
- without the carry rule: exact when no carry is needed; under strict critics, the no-carry bug;
- GPS works units first and answers exactly; without the carry operator it gives up on exactly the carry problems;
- **the same deficit, two regimes**: a confident bug under repair, no answer under GPS.

Instance: four repairs, four bugs on $15 + 17$.

The problem-solving side. In general, for every problem: the bridge to the column backup, place value and the transfer of single-digit errors, and the two results without the carry rule, exactness when no carry is needed and the no-carry bug under strict critics. Also for every problem now: GPS's units-first order and exact answers, its giving up on exactly the carry problems when the carry operator is missing, and the divergence of the two regimes on the same deficit. The four bugs on fifteen plus seventeen remain an instance.

Experiments written against the memory interface

General:

- recency and order-freedom are inconsistent wherever a valid design exists;
- order-freedom gives no order effect, recency a positive one; under forced choice, order-freedom reports 0 however biased the guessing;
- a flat serial-position curve, from certainty and stability alone;
- confident misnaming, derived from EPAM's matrix cells.

Instances: ACT-R's rising curve; MINERVA returns the never-studied prototype; ACT-R says "don't know" more to strangers.

Experiments written against the memory interface

General:

- recency and order-freedom are inconsistent wherever a valid design exists;
- order-freedom gives no order effect, recency a positive one; under forced choice, order-freedom reports 0 however biased the guessing;
- a flat serial-position curve, from certainty and stability alone;
- confident misnaming, derived from EPAM's matrix cells.

Instances: ACT-R's rising curve; MINERVA returns the never-studied prototype; ACT-R says "don't know" more to strangers.

Experiments, written once against the memory interface. In general: recency and order-freedom are inconsistent wherever a valid design exists; the order effects each law predicts, including under a forced-choice response model; a flat serial-position curve from two laws alone; and confident misnaming of faces derived from EPAM's cells in the matrix. As instances, each family's own behavior on the shared experiments.

Concepts and semantic memory

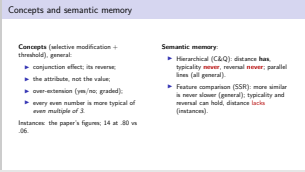
Concepts (selective modification + threshold), general:

- conjunction effect; its reverse;
- the attribute, not the value;
- over-extension (yes/no; graded);
- every even number is more typical of *even multiple of 3*.

Instances: the paper's figures; 14 at .80 vs .06.

Semantic memory:

- Hierarchical (C&Q): distance **has**, typicality **never**, reversal **never**; parallel lines (all general).
- Feature comparison (SSR): more similar is never slower (general); typicality and reversal can hold, distance lacks (instances).



Concepts and semantic memory. For selective modification with threshold membership, the conjunction effect, its reverse, the attribute-not-value prediction and over-extension are general, as is the theorem that every even number is more typical of even multiple of three. The paper's figures and the fourteen case are instances. For semantic memory, Collins and Quillian's verdicts are all general, for every net; the feature comparison model's monotonicity is general, but its typicality effect, reversal and lack of a distance effect are shown in one feature set.

View application

General:

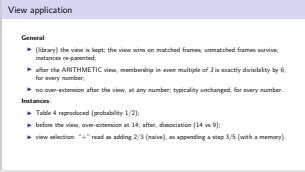
- (library) the view is kept; the view wins on matched frames; unmatched frames survive; instances re-parented;
- after the ARITHMETIC view, membership in *even multiple of 3* is exactly divisibility by 6, for every number;
- no over-extension after the view, at any number; typicality unchanged, for every number.

Instances:

- Table 4 reproduced (probability 1/2);
- before the view, over-extension at 14; after, dissociation (14 vs 9);
- view selection: "+" read as adding 2/3 (naive), as appending a step 3/5 (with a memory).

What CogLean proves

View application



View application. In general: the library's four properties of the merge, and, after the arithmetic view, exact membership, no over-extension and unchanged typicality, each for every number. As instances: the paper's Table 4, the over-extension before the view, the dissociation after it, and the two readings of the plus key under view selection.

Derived, or close to the definitions? (the audit)

Compositional	confident errors (recognition + associations); after the view (view application + categorization); the two regimes diverge
Derived	no-carry bug; GPS order; isolation effect; DOA's counting error share; EPAM's oscillation
Close	serial position (now from a rule); conjunction effect; DOA's closed forms
Definitional	hierarchical network's costs; algorithm properties (soundness, merges)

A judgment, not a check (`notes/EXPLANATORY_AUDIT.md`). Its weakest results are its targets.

Derived, or close to the definitions? (the audit)

Compositional	confident errors (recognition + associations); after the view (view application + categorization); the two regimes diverge
Derived	no-carry bug; GPS order; isolation effect; DOA's counting error share; EPAM's oscillation
Close	serial position (now from a rule); conjunction effect; DOA's closed forms
Definitional	hierarchical network's costs; algorithm properties (soundness, merges)

A judgment, not a check (`notes/EXPLANATORY_AUDIT.md`). Its weakest results are its targets.

Lean checks that each theorem follows from its premises, but not whether the premises already contain the phenomenon. So there's a written audit, a judgment rather than a check. The compositional results, where two mechanisms are needed, are the strongest: confident errors, the arithmetic view's effects, and the divergence of the two control regimes. Several are derived from premises that don't mention them, like the no-carry bug and the isolation effect. Some are close to their definitions, such as the serial position effect, though it now comes from a general rule, and the conjunction effect. And some are definitional, like the hierarchical network's costs. The weakest results are the targets for the next round.

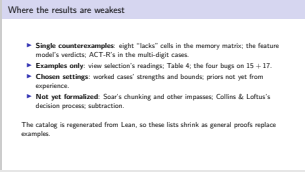
Where the results are weakest

- **Single counterexamples**: eight "lacks" cells in the memory matrix; the feature model's verdicts; ACT-R's in the multi-digit cases.
- **Examples only**: view selection's readings; Table 4; the four bugs on $15 + 17$.
- **Chosen settings**: worked cases' strengths and bounds; priors not yet from experience.
- **Not yet formalized**: Soar's chunking and other impasses; Collins & Loftus's decision process; subtraction.

The catalog is regenerated from Lean, so these lists shrink as general proofs replace examples.

What CogLean proves

Where the results are weakest



Finally, where the results are weakest. Single counterexamples: the eight lacks cells in the memory matrix, and the feature model's and ACT-R's verdicts in their worked cases. Results on examples only: view selection's readings, Table 4, and the four bugs on fifteen plus seventeen. Settings chosen for illustration, where priors should come from experience. And mechanisms not yet formalized: Soar's chunking, Collins and Loftus's decision process, and subtraction. Because the catalog is generated from Lean, these lists will shrink exactly as general proofs replace examples.